Adaptive RDMA Optimization for Distributed LLM Workloads

Update 8: RDMA Tensor Cache Architecture and Neumann Integration

Nathan Gopee
MS Computer Science Candidate
SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

Abstract

Heterogeneous GPU clusters present a precision mismatch problem that fundamentally limits distributed LLM training and inference. This update presents the RDMA Tensor Cache, a centralized precision management layer implemented within the Neumann tensor database, designed for a two-tower home lab: Tower 1 (Ryzen 9 7950X, RTX 5070 Ti 16GB, Windows) and Tower 2 (Proxmox/Linux, 2× Tesla V100 32GB). The V100s lack BF16 support, forcing FP16 computation with its narrow dynamic range; the 5070 Ti supports BF16 natively but runs on Windows without kernel RDMA. We resolve this with an RDMA-accessible FP32 master weight store in Neumann, stochastic rounding for precision-lossy conversions, predictive tensor prefetching, and a centralized KV cache that eliminates the per-rank duplication problem inherent to Multi-Latent Attention (MLA) and Multi-Query Attention (MQA) under tensor parallelism. We detail the `tensor_rdma` crate architecture, present three experimental pillars for validation, and describe integration paths through UCX/UCC, SoftRoCEv2, and the emerging Ultra Ethernet Consortium (UEC) 1.0 standard.

Contents

1 Overview **4**

2 NVIDIA Open-Source Ecosystem **4**

2.1 NCCL 4

2.2 UCX and HPC-X 5

2.3 GPUDirect RDMA and nvidia-peermem 5

2.4 GDRCopy 6

2.5 NIXL 6

2.6 Megatron-LM . . . 6
2.7 InfiniBand RDMA Foundations . . . 6

3 vLLM Parallelism Analysis **7**
3.1 Tensor Parallelism . . . 7
3.2 Pipeline Parallelism . . . 8
3.3 MoE Expert Parallelism . . . 8
3.4 TP vs PP Selection for Our Cluster . . . 8
3.5 The MLA/MQA KV Cache Problem . . . 9

4 RDMA Tensor Cache Architecture **9**
4.1 Precision Pipeline . . . 9
4.2 Stochastic Rounding . . . 11
4.3 Centralized KV Cache . . . 12

5 Neumann Integration **12**
5.1 Module Mapping . . . 12
5.2 Device Capability Model . . . 14
5.3 Transport Abstraction . . . 14

6 Experimental Design: Three Pillars **15**
6.1 Pillar 1: Precision Switching Overhead . . . 15
6.2 Pillar 2: Prefetching Effectiveness . . . 15
6.3 Pillar 3: Adaptive Quantization . . . 16
6.4 Target Performance Summary . . . 17
6.5 Workload Specifications . . . 17

7 Multistage Fine-Tuning and SAE Feature Steering **17**
7.1 Traditional Fine-Tuning Pipeline . . . 18
7.2 Sparse Auto-Encoder Feature Steering . . . 18
7.2.1 Feature Steering . . . 18
7.2.2 Mapping to Neumann . . . 19
7.2.3 Advantages over Gradient-Based Fine-Tuning . . . 19

8 Python Prototype **20**
8.1 Architecture . . . 20
8.2 vLLM Connector . . . 21

9 SoftRoCEv2 Baseline **21**
9.1 Configuration . . . 22
9.2 GPUDirect RDMA Setup . . . 22
9.3 Expected Baseline Measurements . . . 23

9.4 Benchmark Methodology 23

10 UCX/UCC Integration Path **24**

10.1 UCX Transport Mapping 24

10.2 UCC for vLLM Collectives 24

10.3 UEC 1.0: Future RDMA-Native Ethernet 24

10.4 Transport Negotiation Flow 25

1 Overview

This update advances the thesis in three directions. First, we formalize the RDMA Tensor Cache architecture that bridges the precision gap between V100 and 5070 Ti GPUs by centralizing master weights in Neumann's `tensor_rdma` crate. Second, we analyze the parallelism strategies available in vLLM and identify a structural limitation of tensor parallelism for MLA/MQA models that motivates our centralized KV cache design. Third, we introduce Sparse Auto-Encoders (SAEs) as an alternative to gradient-based fine-tuning, showing how SAE feature vectors map naturally to Neumann's `SparseVector` representation with 51× compression.

The home lab consists of two towers connected by Mellanox ConnectX-4 100GbE NICs:

Table 1: Home lab hardware configuration

Component	**Tower 1 (Windows)**	**Tower 2 (Proxmox/Linux)**
CPU	Ryzen 9 7950X (16c/32t)	Xeon (host CPU)
RAM	192 GB DDR5	Host RAM
GPU	RTX 5070 Ti 16 GB (compute 12.0)	2× Tesla V100 32 GB
NIC	Mellanox ConnectX-4 100GbE	Mellanox ConnectX-4 100GbE
OS	Windows 11	Proxmox (Debian/Linux)
RDMA	TCP/gRPC fallback (no kernel RDMA)	SoftRoCEv2 / GPUDirect RDMA
BF16	Native	Not supported
FP16	Supported	Native compute
IP	–	192.168.2.111 (South-Tower)

The fundamental tension is that the V100 is a datacenter GPU with full GPUDirect RDMA support and Tensor Core FP16 compute but no BF16, while the 5070 Ti is a consumer GPU with BF16 native support but no RDMA stack on Windows. The RDMA Tensor Cache resolves this asymmetry by placing FP32 master weights in Neumann, accessible from both towers through different transport paths.

2 NVIDIA Open-Source Ecosystem

The NVIDIA communication stack comprises several interdependent libraries that form the foundation for GPU-accelerated distributed computing. Understanding their boundaries and transport mechanisms is essential for positioning the RDMA Tensor Cache within the ecosystem.

2.1 NCCL

NCCL [11] provides collective communication primitives (AllReduce, AllGather, ReduceScatter, AllToAll) optimized for multi-GPU training. It operates at the collective level and handles topology detection, ring/tree algorithm selection, and NVLink/PCIe/InfiniBand transport. NCCL is tightly coupled to NVIDIA hardware—it uses CUDA kernels for data packing and relies on `cudaIpcGetMemHandle` for intra-node peer-to-peer transfers.

For our cluster, NCCL can drive the V100-to-V100 communication path on Tower 2. However, NCCL cannot bridge heterogeneous precision natively; it transmits tensors in their source format without conversion.

2.2 UCX and HPC-X

UCX (Unified Communication X) [14] is a point-to-point communication framework providing transport-agnostic abstractions over InfiniBand, RoCE, TCP, and shared memory. HPC-X v2.15 [10] bundles UCX with optimized MPI and collective layers. UCX exposes several transport modes relevant to our ConnectX-4 NICs:

Table 2: UCX transport modes for ConnectX-4

Transport	**Mode**	
RC	Reliable Connection	One QP per peer. Guarant
DC	Dynamic Connection	ConnectX-4+. Single QP multi
UD	Unreliable Datagram	Connectionless. Useful
TCP	Socket Fallback	S
SHM	Shared Memory	Intra-noc
CUDA	GPU Memory	Direct GPU buffer

The distinction between datacenter and consumer GPUs is critical here. The V100, as a datacenter-class GPU, supports GPUDirect RDMA: the NIC can DMA directly into GPU memory without staging through host RAM. The 5070 Ti, as a consumer GPU on Windows, must use TCP or gRPC as its transport, with data copied through system memory.

2.3 GPUDirect RDMA and nvidia-peermem

GPUDirect RDMA [18] enables RDMA NICs to read/write GPU memory directly. It requires the `nvidia-peermem` kernel module, which registers GPU memory regions with the InfiniBand subsystem. On Tower 2 (Linux), this is the target configuration for V100 tensor transfers:

Listing 1: GPUDirect RDMA setup on Tower 2

```
# Load nvidia-peermem for GPUDirect RDMA
modprobe nvidia-peermem

# Verify registration
dmesg | grep nv_mem
# nvidia-peermem: module loaded, peer memory registered

# Verify RDMA devices
ibv_devinfo
# hca_id: mlx5_0
#   transport: InfiniBand (0)
```

```
#   fw_ver: 12.28.2006
#   node_guid: ...
#   phys_port_cnt: 1
```

2.4 GDRCopy

GDRCopy [9] provides a low-latency mechanism for CPU-initiated copies to/from GPU memory, complementing GPUDirect RDMA (which is NIC-initiated). For small tensors (control messages, metadata, KV cache headers), GDRCopy avoids the overhead of launching a CUDA kernel for the copy.

2.5 NIXL

NIXL (NVIDIA Inference Xfer Library) [12] provides point-to-point tensor transfer primitives for disaggregated inference. Where NCCL focuses on collective operations for training, NIXL targets the prefill/decode KV cache transfer pattern in serving systems. Its `NixlConnector` interface in vLLM serves as a reference for our middleware integration (Section 8).

2.6 Megatron-LM

Megatron-LM [20] established the canonical model parallelism strategies (tensor parallelism, pipeline parallelism, data parallelism) that all subsequent distributed training frameworks reference. Its tensor parallelism implementation shards weight matrices column-wise or row-wise across GPUs, requiring AllReduce after each transformer layer. This communication pattern directly informs our prefetch engine design.

Megatron-LM's approach to mixed precision training is particularly relevant: it maintains FP32 master weights on the optimizer node while broadcasting FP16 copies to compute nodes. Our RDMA Tensor Cache generalizes this pattern by placing the FP32 master store in Neumann (accessible via RDMA) rather than requiring it to reside in GPU memory on a designated node. This decoupling enables the heterogeneous V100/5070 Ti configuration where no single GPU type can serve as the universal master.

2.7 InfiniBand RDMA Foundations

The RDMA programming model [16] provides three key abstractions that `tensor_rdma`'s transport layer mirrors:

1. **Memory Registration:** Before RDMA operations, memory regions must be registered with the NIC via `ibv_reg_mr`. This pins physical pages and provides the NIC with a translation table. Our `register_buffer` method models this step. Registration is expensive ($\sim$10 μs) but amortized across thousands of RDMA operations on the same buffer.

2. **Queue Pairs (QPs):** Communication endpoints consisting of a Send Queue and Receive Queue. Our `TransportConfig::SoftRoce` configures QP parameters (device, GID index, port). The choice between Reliable Connection (RC) and Dynamic Connection (DC) QPs affects scalability: RC requires $O(n^2)$ QPs for an n-node cluster, while DC multiplexes a single QP across all peers.

3. **One-Sided Operations:** RDMA Read and Write execute without involving the remote CPU. The initiating NIC issues the DMA directly to/from the remote NIC, which accesses the pre-registered memory region. This is why GPUDirect RDMA is so effective: the NIC writes directly into GPU BAR1 memory, bypassing both CPUs entirely. Our `rdma_write` and `rdma_read` methods map to `ibv_post_send` with `IBV_WR_RDMA_WRITE` and `IBV_WR_RDMA_READ` opcodes respectively.

For a tensor of size S bytes, the total transfer time over RDMA is:

$$T_{\text{RDMA}} = T_{\text{latency}} + \frac{S}{B_{\text{link}}} + T_{\text{DMA}} \tag{1}$$

where T_{latency} is the one-way latency ($\sim$2 μs for hardware RoCE), B_{link} is the link bandwidth (12.5 GB/s for 100GbE), and T_{DMA} is the DMA engine overhead ($\sim$0.5 μs). For a 256 MB gradient tensor (typical for a transformer layer with hidden dimension 4096), $T_{\text{RDMA}} \approx 2 + 20480 + 0.5 \approx$ 20.5 ms. This is dominated by bandwidth, not latency, confirming that 100GbE is the bottleneck rather than RDMA protocol overhead.

3 vLLM Parallelism Analysis

The choice between tensor parallelism (TP) and pipeline parallelism (PP) has significant implications for RDMA communication patterns and KV cache management. We analyze the trade-offs based on vLLM's architecture and community discussions [22].

3.1 Tensor Parallelism

Tensor parallelism partitions weight matrices across GPUs within a single node. Each GPU computes a shard of the attention and MLP layers, then synchronizes via AllReduce. The communication volume per layer is:

$$V_{\text{TP}} = 2 \cdot b \cdot s \cdot h \cdot (\text{TP} - 1)/\text{TP} \tag{2}$$

where b is batch size, s is sequence length, h is hidden dimension, and TP is the tensor parallel degree. This AllReduce occurs *within* each transformer layer, making it latency-sensitive: any delay propagates through all layers.

For Tower 2 with 2$\times$ V100, TP=2 is natural. The two GPUs communicate via PCIe (no NVLink on our configuration), so AllReduce latency is dominated by PCIe bandwidth ($\sim$12 GB/s bidirectional per GPU).

3.2 Pipeline Parallelism

Pipeline parallelism assigns contiguous blocks of transformer layers to different GPUs. Communication occurs only at pipeline stage boundaries, transmitting activations forward and gradients backward. The communication volume per micro-batch is:

$$V_{\text{PP}} = b \cdot s \cdot h \tag{3}$$

which is independent of the parallelism degree. However, pipeline parallelism introduces the *pipeline bubble*—time during which GPUs are idle waiting for upstream stages to complete. The bubble fraction for a p-stage pipeline with m micro-batches is:

$$\text{Bubble} = \frac{p-1}{m+p-1} \tag{4}$$

For cross-node communication (Tower 1 to Tower 2), PP is preferable because it moves communication out of the critical per-layer path. The 100GbE link has higher latency than intra-node PCIe but sufficient bandwidth for activation transfers.

3.3 MoE Expert Parallelism

For Mixture-of-Experts models, expert parallelism (EP) distributes experts across GPUs. The AMD ROCm guide [2] specifies the relationship:

$$\texttt{EP_SIZE} = \texttt{TP} \times \texttt{DP} \tag{5}$$

Expert parallelism uses AllToAll communication to route tokens to their assigned experts. This creates a distinct traffic pattern: rather than the synchronous AllReduce of TP, AllToAll is inherently asymmetric with variable-size messages.

3.4 TP vs PP Selection for Our Cluster

Table 3: Parallelism strategy comparison for the two-tower cluster

Property	**Tensor Parallelism**	**Pipeline Parallelism**
Communication pattern	AllReduce per layer	Activation transfer per stage
Communication frequency	$2L$ per forward pass	$p-1$ per micro-batch
Latency sensitivity	High (synchronous)	Low (async pipeline)
Bandwidth requirement	$O(b \cdot s \cdot h)$ per layer	$O(b \cdot s \cdot h)$ per stage
Suitable interconnect	NVLink, PCIe	Ethernet, RDMA
Our intra-Tower 2	V100↔V100 (PCIe)	Not needed (2 GPUs)
Our cross-tower	Not recommended	Tower 1↔Tower 2

For our configuration, the optimal strategy is TP=2 within Tower 2 (V100 pair over PCIe)

combined with PP across towers (100GbE RDMA). The 5070 Ti on Tower 1 handles a separate pipeline stage, receiving activations from Tower 2 and returning results. This minimizes cross-tower synchronous communication while exploiting the high-bandwidth PCIe path between V100s.

3.5 The MLA/MQA KV Cache Problem

Multi-Latent Attention (MLA, as in DeepSeek-V2 [5]) and Multi-Query Attention (MQA) reduce KV cache memory by sharing a single key-value head across all attention heads. This creates a structural incompatibility with tensor parallelism: the KV cache cannot be sharded along the head dimension because there is only one head.

Under standard TP, each GPU stores a partition of the KV cache corresponding to its attention head shard. With $h_{\text{kv}} = 1$ (MQA) or $h_{\text{kv}} = 1$ (MLA's compressed latent), each rank must maintain a *full copy* of the KV cache. For a model with context length C, hidden dimension d, and L layers:

$$\text{KV Memory (per rank, MHA)} = \frac{2 \cdot L \cdot C \cdot d}{\text{TP}} \tag{6}$$

$$\text{KV Memory (per rank, MQA)} = 2 \cdot L \cdot C \cdot d_{\text{kv}} \quad \text{(no reduction with TP)} \tag{7}$$

This duplication wastes VRAM and increases the memory pressure on each GPU. For our V100 pair with 32 GB each, KV cache duplication can consume a significant fraction of available memory.

Challenge 1. MLA and MQA models cannot benefit from tensor parallelism for KV cache distribution. Each TP rank stores a full copy of the KV cache, negating the memory savings that motivated these attention variants.

Our solution is to externalize the KV cache to Neumann, accessed via RDMA. A single centralized copy serves all TP ranks through one-sided RDMA reads, eliminating duplication. Neumann's `rdma_cache` module already provides the buffer registration and remote access semantics required (Section 5).

4 RDMA Tensor Cache Architecture

The RDMA Tensor Cache mediates all tensor transfers between Tower 1 and Tower 2, handling precision conversion, gradient accumulation, and KV cache management. Figure 1 shows the overall system architecture.

4.1 Precision Pipeline

The core challenge is that FP16 (used by V100 Tensor Cores) has 5 exponent bits and 10 mantissa bits, giving a representable range of $[5.96 \times 10^{-8},\ 65504]$. During training, gradient values routinely fall outside this range, causing overflow (gradients > 65504 become `inf`) and underflow (gradients $< 6 \times 10^{-8}$ become zero). BF16 shares FP32's 8 exponent bits, matching its dynamic range of $[1.18 \times 10^{-38},\ 3.4 \times 10^{38}]$, but is unavailable on V100.

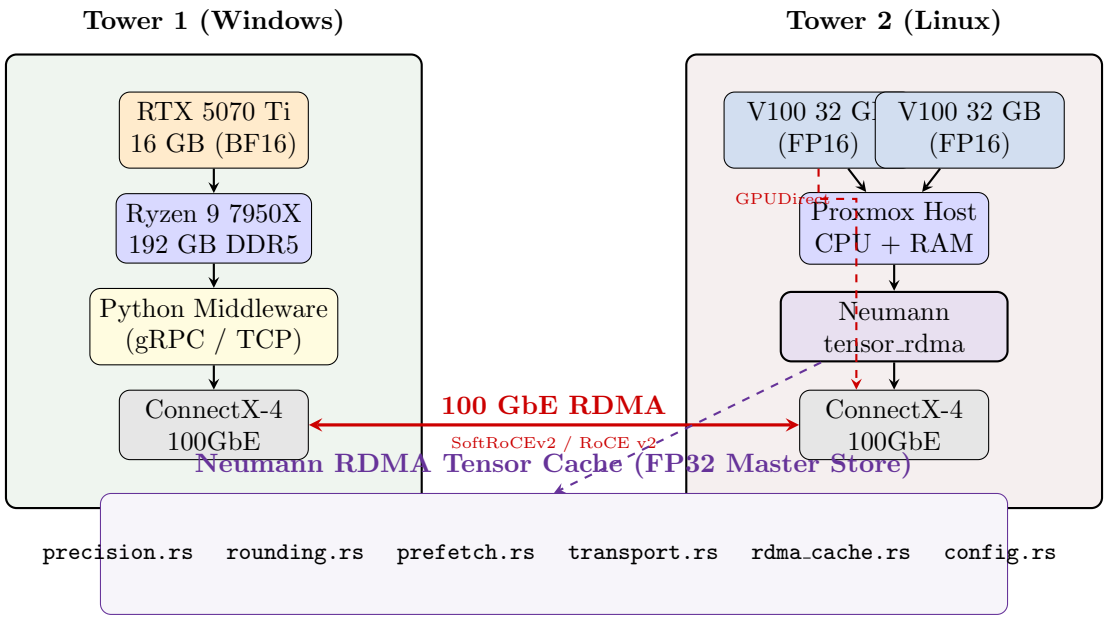

Figure 1: Two-tower system architecture. Neumann's `tensor_rdma` crate runs on Tower 2, serving as the centralized FP32 master weight store. Tower 1 accesses tensors via gRPC/TCP fallback; Tower 2's V100s access them via GPUDirect RDMA through `nvidia-peermem`.

Table 4: Floating-point format comparison for LLM training

Format	**Exponent**	**Mantissa**	**Range**	**Bytes**	**GPU Support**
FP32	8	23	$\pm 3.4 \times 10^{38}$	4	All
BF16	8	7	$\pm 3.4 \times 10^{38}$	2	Ampere+, not V100
FP16	5	10	$\pm 6.5 \times 10^{4}$	2	All (Tensor Cores)
MXFP4	2	1	± 6	0.5	Blackwell

The critical observation is that BF16 and FP32 share the same exponent width (8 bits), giving BF16 the full FP32 dynamic range at half the memory. FP16's 5-bit exponent limits its range to 6.5×10^4, which is routinely exceeded during training. Loss scaling partially mitigates this but adds implementation complexity and can mask genuine numerical issues.

The RDMA Tensor Cache resolves this through a three-stage precision pipeline:

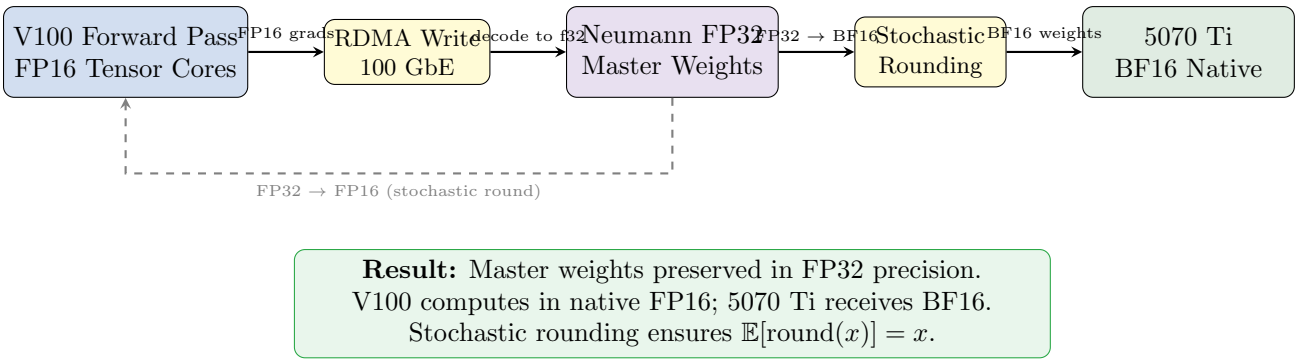

Figure 2: Precision pipeline. Gradients computed in V100 FP16 are accumulated into FP32 master weights stored in Neumann. Stochastic rounding converts to device-native precision for each GPU.

4.2 Stochastic Rounding

Standard rounding (round-to-nearest-even) introduces a systematic bias when converting FP32 values to reduced precision: values exactly between two representable numbers always round to the even option, causing certain gradient directions to be preferred. Stochastic rounding eliminates this bias by making the rounding direction probabilistic.

For an FP32 value x being rounded to a lower-precision format with adjacent representable values $\lfloor x \rfloor$ and $\lceil x \rceil$:

$$P(\text{round up}) = \frac{x - \lfloor x \rfloor_{\text{fp16}}}{\lceil x \rceil_{\text{fp16}} - \lfloor x \rfloor_{\text{fp16}}} \tag{8}$$

This gives the unbiasedness property:

$$\mathbb{E}[\text{round}(x)] = \lfloor x \rfloor \cdot (1 - P) + \lceil x \rceil \cdot P = x \tag{9}$$

which ensures that gradient accumulation over many steps converges to the correct value despite precision loss in individual steps [7].

In the `tensor_rdma` crate, stochastic rounding is implemented in `rounding.rs` using the `half` crate for FP16/BF16 manipulation and `rand` for probability generation. The `next_fp16` and `next_bf16` helper functions find adjacent representable values via bit manipulation:

Listing 2: Stochastic rounding implementation sketch (from rounding.rs)

```
pub fn stochastic_round_f32_to_f16(value: f32, rng: &mut impl Rng) -> f16
    {
```

```
    let floor = f16::from_f32(value);
    let floor_f32 = floor.to_f32();

    if (floor_f32 - value).abs() < f32::EPSILON {
        return floor; // Exact representation
    }

    let ceil = next_fp16(floor, value > floor_f32);
    let ceil_f32 = ceil.to_f32();
    let gap = ceil_f32 - floor_f32;
    let fraction = (value - floor_f32) / gap;

    if rng.gen::<f32>() < fraction { ceil } else { floor }
}
```

4.3 Centralized KV Cache

The MLA/MQA duplication problem identified in Section 3.5 is resolved by storing the KV cache in Neumann's `RdmaTensorCache`, keyed by layer index and sequence position. Each TP rank issues one-sided RDMA reads to fetch the KV entries it needs for its attention head shard, rather than maintaining a local copy.

The RDMA read pattern for KV cache access is:

1. Prefetch engine predicts which layer/position entries will be needed next based on the current decoding step.
2. Transport layer issues RDMA Read to Neumann's registered buffer for the predicted entries.
3. Attention computation proceeds on local GPU memory with the fetched KV data.
4. New KV entries generated during decoding are written back via RDMA Write.

For a model with L layers and context length C, this reduces total KV memory across TP ranks from $\text{TP} \times 2LC \cdot d_{\text{kv}}$ to $2LC \cdot d_{\text{kv}} + \text{TP} \times \text{prefetch_buffer}$. The prefetch buffer is typically 3–5 layers deep, a small fraction of the total cache.

5 Neumann Integration

The `tensor_rdma` crate [1] is a workspace member in the Neumann unified tensor runtime. It depends on `tensor_store` for local key-value storage and `tensor_compress` for delta encoding. Figure 3 shows the dependency structure.

5.1 Module Mapping

Table 5 maps thesis concepts to their Rust implementations.

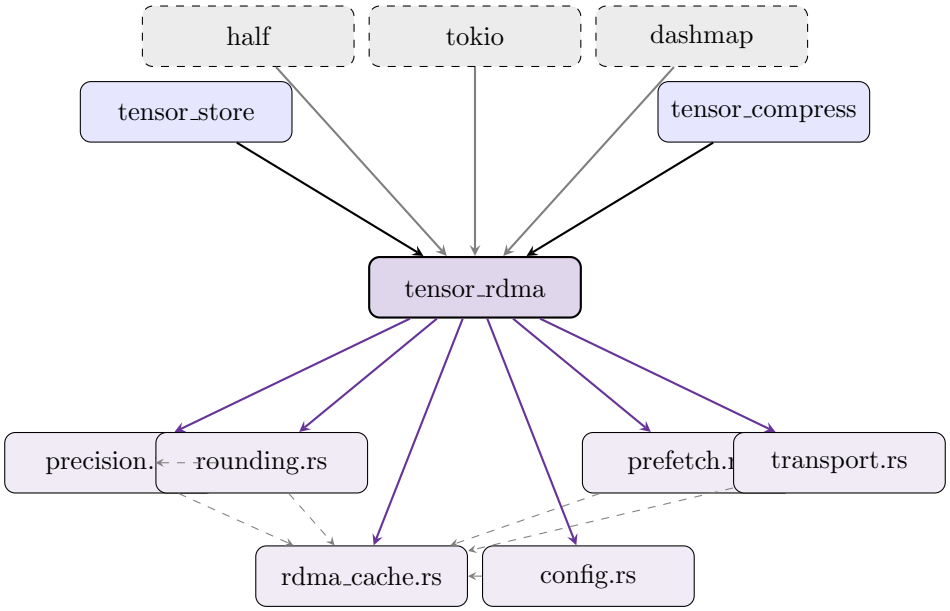

Figure 3: Neumann `tensor_rdma` crate dependency graph. The crate depends on `tensor_store` for local caching and `tensor_compress` for delta encoding. Six internal modules implement the RDMA Tensor Cache.

Table 5: Thesis concepts mapped to `tensor_rdma` modules

Thesis Concept	**Module**	
Precision management	`precision.rs`	`PrecisionFormat` (FP16, BF16, F
Stochastic rounding	`rounding.rs`	`s`
Predictive prefetching	`prefetch.rs`	`PrefetchEngine`, `Acce`
RDMA transport	`transport.rs`	`RdmaTransport` tra
Tensor cache	`rdma_cache.rs`	`RdmaTens`
Configuration	`config.rs`	`RdmaCacheConfig`, `PrefetchS`

5.2 Device Capability Model

The `DeviceCapability` struct encodes what each GPU supports. The `PrecisionRouter` uses these to select wire formats:

Listing 3: Device capabilities for our hardware

```
// V100: FP16 native, FP32 supported, no BF16, GPUDirect RDMA
DeviceCapability::v100_32gb() -> DeviceCapability {
    name: "Tesla V100 32GB",
    supported_formats: [FP16, FP32],
    native_compute: FP16,
    vram_bytes: 34_359_738_368,
    gpu_direct_rdma: true,
}

// RTX 5070 Ti: BF16 native, FP16/FP32 supported, no GPUDirect
DeviceCapability::rtx5070ti() -> DeviceCapability {
    name: "RTX 5070 Ti 16GB",
    supported_formats: [FP16, BF16, FP32],
    native_compute: BF16,
    vram_bytes: 17_179_869_184,
    gpu_direct_rdma: false,
}
```

The `PrecisionRouter::common_format()` method determines the lowest common denominator: for V100 + 5070 Ti, this is FP16 (since V100 lacks BF16). However, the cache stores in FP32 and converts per-device, avoiding the precision ceiling that a common format would impose.

5.3 Transport Abstraction

The `RdmaTransport` trait defines four core operations modeled on RDMA verbs semantics:

1. `register_buffer`: Analogous to `ibv_reg_mr`. Pins memory and returns a `BufferHandle` with a remote key.
2. `deregister_buffer`: Releases the memory registration.
3. `rdma_write`: One-sided write to a remote buffer at a specified offset. No notification to the remote CPU.
4. `rdma_read`: One-sided read from a remote buffer. Data is pulled without remote CPU involvement.

Three implementations exist: `MemoryTransport` for unit testing (shared `HashMap` simulating RDMA), `TcpFallbackTransport` for Windows (Tower 1), and the planned `SoftRoceTransport` for Linux (Tower 2, feature-gated behind `rdma-core`).

6 Experimental Design: Three Pillars

We structure validation around three experimental pillars, each targeting a distinct aspect of the RDMA Tensor Cache.

6.1 Pillar 1: Precision Switching Overhead

Hypothesis: Stochastic rounding adds less than 5% overhead compared to deterministic truncation, while eliminating the gradient bias that causes training divergence.

Methodology:

1. Generate synthetic gradient tensors of varying sizes (10^3 to 10^8 elements) spanning the FP16 representable range.
2. Measure wall-clock time for FP16 → FP32 decode, FP32 weight update, and FP32 → FP16 stochastic round.
3. Compare against deterministic round-to-nearest on the same tensors.
4. Compute bias: $\text{Bias} = \frac{1}{N}\sum_{i=1}^{N}(\text{round}(x_i) - x_i)$ over 10^6 samples.

Metrics:

- Conversion throughput (GB/s) for each direction: FP16↔FP32, FP32↔BF16
- Statistical bias magnitude (should be $< 10^{-6}$ for stochastic, $> 10^{-4}$ for deterministic)
- Overhead ratio: $T_{\text{stochastic}}/T_{\text{deterministic}}$

6.2 Pillar 2: Prefetching Effectiveness

Hypothesis: The adaptive prefetch engine achieves >90% hit rate for sequential inference workloads and >70% for fine-tuning workloads, hiding RDMA latency for the majority of tensor accesses.

Methodology:

1. Trace tensor access patterns during Llama-7B inference on a single V100 (sequential layer-by-layer).
2. Trace access patterns during LoRA fine-tuning (forward sequential, backward reverse, optimizer random).
3. Replay traces through the prefetch engine with varying depths (1, 3, 5, 10).
4. Measure hit rate: fraction of accesses where the tensor was already prefetched.

Metrics:

- Prefetch hit rate by strategy (Sequential, Strided, LayerSweep, Adaptive)

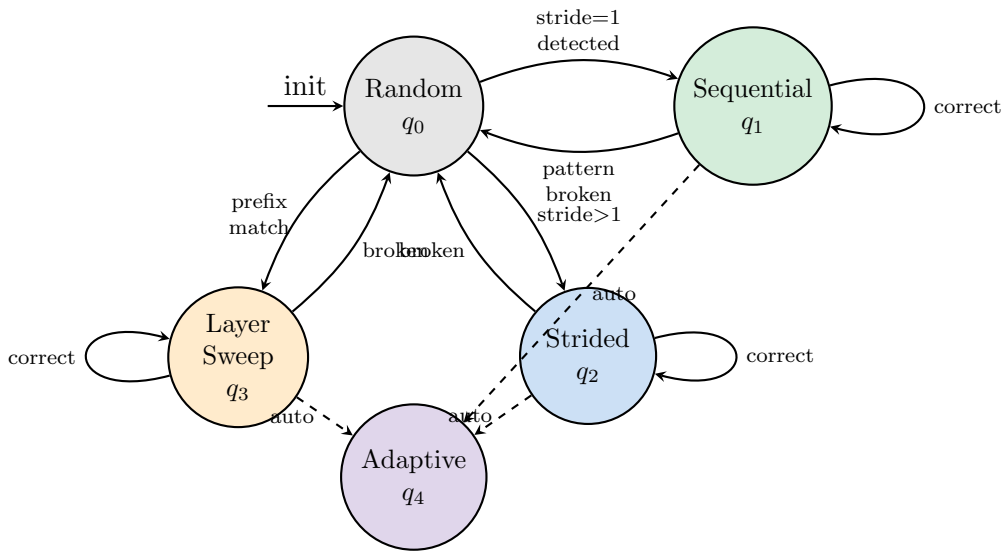

Figure 4: Prefetch engine state machine. The `Adaptive` strategy delegates to the detected pattern. State transitions occur when the `classify_pattern` function analyzes the access history buffer (1024 entries).

- Latency breakdown: hit path (<1 μs local lookup) vs. miss path (190 μs SoftRoCE / 2 μs hardware RoCE)
- Wasted prefetches: tensors prefetched but never used (bandwidth waste)

6.3 Pillar 3: Adaptive Quantization

Hypothesis: MXFP4 quantization [19] achieves 4× bandwidth reduction with less than 1% accuracy loss for inference-time KV cache transfers, and stochastic rounding preserves gradient quality under 2× FP16 bandwidth reduction.

Methodology:

1. Measure inference quality (perplexity on WikiText-2) with KV cache stored in FP32, FP16, BF16, and MXFP4.
2. Measure training convergence (loss curve) with gradients transferred in FP32, FP16 (deterministic), and FP16 (stochastic).
3. Profile 100GbE utilization under each precision configuration.

Metrics:

- Perplexity delta: $\Delta\text{PPL} = \text{PPL}_{\text{quantized}} - \text{PPL}_{\text{FP32}}$
- Bandwidth savings: bytes transferred per training step
- Convergence rate: steps to reach target loss

6.4 Target Performance Summary

Table 6: Target performance metrics across all three pillars

Pillar	Metric	Baseline	Target
Precision	Stochastic round overhead	–	$< 5\%$
	Bias (stochastic)	–	$< 10^{-6}$
	Conversion throughput	–	> 50 GB/s
Prefetching	Hit rate (inference)	0%	$> 90\%$
	Hit rate (fine-tuning)	0%	$> 70\%$
	Wasted prefetch ratio	–	$< 15\%$
Quantization	ΔPPL (FP16 KV)	FP32	< 0.1
	ΔPPL (MXFP4 KV)	FP32	< 0.5
	Bandwidth reduction	1×	2–4×

The three pillars are designed to be independently measurable but collectively demonstrate the viability of the RDMA Tensor Cache for production heterogeneous clusters. Pillar 1 validates the precision management layer in isolation; Pillar 2 validates the prefetch engine's ability to hide network latency; Pillar 3 demonstrates end-to-end quality preservation under aggressive bandwidth reduction.

6.5 Workload Specifications

The experimental workloads target models that exercise the full precision pipeline:

1. **Llama 2 7B (inference):** Sequential layer-by-layer access pattern. KV cache grows linearly with generated tokens. Tests Pillar 2 (sequential prefetch) and Pillar 3 (KV cache quantization).

2. **Mistral 7B with Sliding Window (inference):** Strided access pattern due to sliding window attention. KV cache has fixed maximum size. Tests Pillar 2 (strided prefetch detection).

3. **DeepSeek-V2 Lite (inference):** MLA attention with compressed latent representation. Single KV head, directly exercises the centralized KV cache design from Section 3.5.

4. **Llama 2 7B LoRA (fine-tuning):** Forward (sequential), backward (reverse sequential), optimizer (random access). Tests all three prefetch strategies and the full gradient→master weight→device weight pipeline.

7 Multistage Fine-Tuning and SAE Feature Steering

We investigate two approaches to adapting LLMs on our heterogeneous cluster: traditional gradient-based fine-tuning with the RDMA Tensor Cache, and SAE-based feature steering as a gradient-free alternative.

7.1 Traditional Fine-Tuning Pipeline

The heterogeneous training pipeline assigns roles based on GPU capabilities:

- **V100 pair (Tower 2):** Forward and backward passes in FP16 Tensor Cores. Gradients are produced in FP16 and transferred to Neumann via RDMA.
- **Neumann (Tower 2 host):** Accumulates gradients into FP32 master weights. Applies optimizer updates (Adam) in full precision.
- **5070 Ti (Tower 1):** Receives BF16 weights via TCP/gRPC for inference validation during training. Can also run the optimizer in BF16 for tasks where BF16 dynamic range is sufficient.

This architecture mirrors the Hetis system [15], which demonstrated 2.25× throughput improvement by assigning distinct stages of the training pipeline to different GPU types. In Hetis, heterogeneous GPUs handle prefill and decode stages separately. We adapt this idea to separate compute (V100 FP16) from precision management (Neumann FP32) and validation (5070 Ti BF16).

7.2 Sparse Auto-Encoder Feature Steering

Sparse Auto-Encoders (SAEs) offer an alternative to gradient-based fine-tuning for modifying model behavior. Instead of adjusting model weights through backpropagation, SAEs decompose model activations into interpretable features that can be directly manipulated [4].

An SAE decomposes a model's internal activation vector $\mathbf{a} \in \mathbb{R}^d$ into a sparse feature vector $\mathbf{f} \in \mathbb{R}^D$ where $D \gg d$:

$$\mathbf{f} = \mathrm{ReLU}(W_{\mathrm{enc}} \cdot \mathbf{a} + \mathbf{b}_{\mathrm{enc}}) \tag{10}$$

$$\hat{\mathbf{a}} = W_{\mathrm{dec}} \cdot \mathbf{f} + \mathbf{b}_{\mathrm{dec}} \tag{11}$$

The key property is that $\mathbf{f}$ is sparse: only $k \ll D$ features are active for any given input. Cunningham et al. [4] showed that these features correspond to interpretable concepts (e.g., "mentions of the Eiffel Tower," "Python code patterns," "formal writing style").

7.2.1 Feature Steering

Given a trained SAE, model behavior can be modified by clamping or scaling specific features. To steer the model toward producing outputs related to feature i:

$$\mathbf{f}'_i = \alpha \cdot \mathbf{f}_{\mathrm{max},i} \tag{12}$$

where α controls the steering strength and $\mathbf{f}_{\mathrm{max},i}$ is the maximum activation observed for feature i during training. The modified activation is then:

$$\mathbf{a}' = W_{\mathrm{dec}} \cdot \mathbf{f}' + \mathbf{b}_{\mathrm{dec}} \tag{13}$$

This modification is applied at inference time with no gradient computation, no backward pass, and no weight updates. The Eiffel Tower Llama demo [17] demonstrates this: clamping the "Eiffel Tower" feature causes Llama to mention the Eiffel Tower in every response regardless of the prompt.

7.2.2 Mapping to Neumann

SAE feature vectors are a natural fit for Neumann's `SparseVector` type, which stores only non-zero indices and values:

Listing 4: SAE features as Neumann SparseVectors

```
// A 65536-dim SAE feature vector with ~50 active features
// Dense: 65536 * 4 bytes = 256 KB
// SparseVector: ~50 * (4 + 4) bytes = 400 bytes
// Compression: 51x (640:1 ratio)

let feature_vector = SparseVector::from_pairs(
    vec![(1204, 0.85), (3891, 0.42), (15023, 0.91), ...],
    65536, // total dimensions
);

// Store in Neumann for RDMA access
store.put("sae:layer_12:eiffel_tower", feature_vector);
```

The 51× compression from dense to sparse representation means SAE feature vectors are trivially transferable over RDMA—a single feature vector fits in a single RDMA read of ∼400 bytes, well below the MTU.

OpenAI's work on scaling SAEs [6] and the HuggingFace SAE collection [3] provide pre-trained SAEs for several open-weight models. These can be loaded into Neumann and served to both towers for inference-time steering.

7.2.3 Advantages over Gradient-Based Fine-Tuning

Table 7: Comparison of fine-tuning approaches for our hardware

Property	**Gradient-Based (LoRA)**	**SAE Steering**
Requires backward pass	Yes	No
GPU memory for optimizer	2–3× model size	Zero
Training time	Hours to days	Seconds (feature lookup)
Precision sensitivity	High (requires FP32 master)	Low (sparse vector ops)
Cross-tower RDMA traffic	Gradients per step	Feature vectors once
Reversibility	Requires separate checkpoint	Immediate (set $\alpha = 0$)
Interpretability	Opaque weight deltas	Named features

SAE steering does not replace fine-tuning for all tasks. It cannot learn genuinely new capabilities, only amplify or suppress existing features. However, for behavioral adaptation (style, tone, topic focus) it eliminates the entire precision management pipeline and reduces RDMA traffic to a single sparse vector read.

8 Python Prototype

The Python prototype bridges the `tensor_rdma` Rust crate with vLLM's distributed inference engine. It follows the middleware architecture established in Update 6 and extends it with RDMA-native connectors.

8.1 Architecture

Listing 5: RDMA Tensor Cache middleware extension

```python
class RDMATensorCacheMiddleware:
    """Extends the ScuffedRDMA middleware with Neumann tensor cache access."""

    def __init__(self, config: MiddlewareConfig):
        self.transport = self._init_transport(config)
        self.neumann_client = NeumannClient(
            host=config.neumann_host,  # 192.168.2.111
            port=config.neumann_port,  # 4791
        )
        self.precision_router = PrecisionRouter(config.devices)

    def get_tensor(self, key: str, device: DeviceCapability) -> torch.Tensor:
        """Fetch tensor from Neumann with automatic precision conversion."""
        wire_format = self.precision_router.optimal_wire_format(device)
        raw = self.neumann_client.rdma_read(key)
        return self._decode_to_torch(raw, wire_format, device)

    def put_gradient(self, key: str, gradient: torch.Tensor,
                     source_device: DeviceCapability) -> None:
        """Send gradient to Neumann for FP32 master weight update."""
        wire_data = self._encode_from_torch(gradient, source_device)
        self.neumann_client.rdma_write(key, wire_data)

    def get_kv_cache(self, layer_idx: int, seq_positions: List[int]
                     ) -> torch.Tensor:
        """Fetch KV cache entries from centralized Neumann store."""
```

```
        keys = [f"kv:layer_{layer_idx}:pos_{p}" for p in seq_positions]
        return self.neumann_client.batch_read(keys)
```

8.2 vLLM Connector

The connector follows vLLM's existing patterns for distributed KV cache management. vLLM supports pluggable connectors (MooncakeConnector [8], NixlConnector [12]) through its transfer engine interface. Our connector implements the same interface:

Listing 6: vLLM RDMA connector skeleton

```
class NeumannKVConnector:
    """vLLM KV cache connector backed by Neumann RDMA Tensor Cache."""

    def __init__(self, vllm_config, role: str):
        self.role = role  # "prefill" or "decode"
        self.middleware = RDMATensorCacheMiddleware(
            MiddlewareConfig.from_vllm(vllm_config)
        )

    def send_kv_caches_and_hidden_states(
        self, model_executable, model_input, kv_caches, hidden_states
    ):
        """Push KV caches to Neumann after prefill."""
        for layer_idx, kv in enumerate(kv_caches):
            key = f"kv:req_{model_input.request_id}:layer_{layer_idx}"
            self.middleware.neumann_client.rdma_write(key, kv)

    def recv_kv_caches_and_hidden_states(self, model_executable,
       model_input):
        """Pull KV caches from Neumann for decode."""
        kv_caches = []
        for layer_idx in range(model_executable.num_layers):
            key = f"kv:req_{model_input.request_id}:layer_{layer_idx}"
            kv = self.middleware.get_kv_cache(layer_idx, model_input.
               positions)
            kv_caches.append(kv)
        return kv_caches, None
```

9 SoftRoCEv2 Baseline

Before deploying hardware RoCE with the ConnectX-4 NICs, we establish a SoftRoCEv2 baseline for Tower 1 ↔ Tower 2 communication. SoftRoCEv2 implements the RDMA verbs API in software over

standard Ethernet, enabling development and debugging of RDMA code paths without specialized hardware configuration.

9.1 Configuration

On Tower 2 (Proxmox/Linux), SoftRoCEv2 is configured through the `rdma_rxe` kernel module:

Listing 7: SoftRoCEv2 setup on Tower 2

```
# Load the software RDMA provider
modprobe rdma_rxe

# Create RXE device over the physical NIC
rdma link add rxe0 type rxe netdev ens3f0

# Verify
rdma link show
# link rxe0/1 state ACTIVE physical_state LINK_UP netdev ens3f0

ibv_devinfo -d rxe0
# hca_id: rxe0
# transport: InfiniBand (0)
# fw_ver: 0.0.0
# max_qp: 1024
# max_mr_size: 0xffffffffffffffff
```

9.2 GPUDirect RDMA Setup

For the V100 GPUs on Tower 2, GPUDirect RDMA provides the most efficient data path. With `nvidia-peermem` loaded, the RDMA NIC can read/write GPU memory directly:

Listing 8: GPUDirect RDMA verification

```
# Load nvidia-peermem
modprobe nvidia-peermem

# Verify GPU BAR1 memory is accessible
nvidia-smi -q | grep "BAR1"
#   BAR1 Memory Usage
#       Total: 32768 MiB
#       Used: 2 MiB
#       Free: 32766 MiB

# Test with perftest
ib_write_bw -d rxe0 --use_cuda=0  # Server (V100 GPU 0)
ib_write_bw -d rxe0 --use_cuda=0 192.168.2.111  # Client
```

9.3 Expected Baseline Measurements

Table 8: Expected SoftRoCEv2 baseline performance (Tower 1 ↔ Tower 2)

Metric	**SoftRoCEv2**	**Hardware RoCE (target)**	**Improvement**
Latency (1 byte)	∼190 μs	∼2 μs	95×
Bandwidth (large msg)	∼8 Gbps	∼90 Gbps	11×
CPU overhead	High (software path)	Near-zero (NIC offload)	–
GPUDirect RDMA	Via host bounce	Direct NIC→GPU	–

SoftRoCEv2 exercises the same verbs API as hardware RoCE, so code developed against it requires no changes when transitioning to hardware. The primary limitations are latency (dominated by kernel context switches) and bandwidth (limited by CPU processing of RDMA protocol headers).

9.4 Benchmark Methodology

The SoftRoCEv2 baseline will be established using the standard `perftest` suite and custom tensor transfer benchmarks:

1. **Microbenchmarks (perftest):**
   - `ib_write_lat`: RDMA Write latency for message sizes 1 B to 4 MB.
   - `ib_read_lat`: RDMA Read latency (same range).
   - `ib_write_bw`: Sustained write bandwidth with 64 KB to 16 MB messages.
   - `ib_read_bw`: Sustained read bandwidth.
2. **Tensor transfer benchmarks:**
   - Round-trip latency for gradient tensors of sizes matching common transformer layers (1024×1024, 4096×4096, 8192×8192).
   - Sustained throughput for streaming weight downloads (simulating model loading via RDMA).
   - Concurrent transfer scalability: 1, 2, 4, 8 simultaneous QPs.
3. **GPUDirect RDMA benchmarks:**
   - CPU-staged vs. GPUDirect latency and throughput comparison on Tower 2.
   - GPU memory registration overhead (`ibv_reg_mr` with `nvidia-peermem`).
   - Effect of GPU memory allocation strategy (`cudaMalloc` vs. `cudaMallocManaged`).

These measurements establish the performance envelope for each transport path, informing the prefetch engine's latency estimates and the middleware's transport selection heuristics.

10 UCX/UCC Integration Path

10.1 UCX Transport Mapping

The middleware's transport abstraction maps directly to UCX transport layers. Table 9 shows the correspondence:

Table 9: Middleware transport mapping to UCX

Middleware Transport	UCX Transport (UCX_TLS)	Use Case
`TcpFallbackTransport`	`tcp`	Tower 1 (Windows, no RDMA)
`SoftRoceTransport`	`rc` (over rxe0)	Development, SoftRoCEv2 baseline
Hardware RoCE (planned)	`rc` or `dc` (over mlx5_0)	Production 100GbE path
GPUDirect RDMA	`cuda_copy,gdr_copy`	V100 direct GPU memory access
Intra-node	`shm,cma`	V100-to-V100 within Tower 2

10.2 UCC for vLLM Collectives

UCC (Unified Collective Communication) [13] provides collective operations (AllReduce, AllGather, AllToAll) built on top of UCX point-to-point primitives. A recent RFC for vLLM [23] proposes replacing NCCL with UCC for environments where NCCL's CUDA dependency is a limitation.

For our cluster, UCC would provide several advantages:

1. UCX transport negotiation handles the heterogeneous path (TCP for Tower 1, RoCE for Tower 2) automatically via `UCX_TLS=rc,tcp`.
2. UCC AllReduce can operate on pre-registered RDMA buffers, avoiding the copy overhead of NCCL's `cudaMallocAsync` path.
3. UCC supports non-CUDA backends, enabling future AMD GPU or CPU-only node integration.

10.3 UEC 1.0: Future RDMA-Native Ethernet

The Ultra Ethernet Consortium (UEC) 1.0 specification [21] defines a new transport protocol designed for AI/HPC workloads. Unlike RoCE, which layers RDMA semantics over Ethernet with several fragile assumptions (lossless fabric, PFC, ECN), UEC provides:

- Native multipath with packet spraying across parallel links
- Ordered delivery without head-of-line blocking
- Built-in congestion control optimized for incast patterns (AllReduce)
- Selective retransmission without connection state reset

UEC is relevant to the thesis timeline because Mellanox (now NVIDIA Networking) ConnectX NICs are expected to receive UEC firmware updates, meaning our ConnectX-4 hardware may gain UEC support without NIC replacement. The middleware's transport abstraction is designed to accommodate UEC as an additional transport backend when it becomes available.

10.4 Transport Negotiation Flow

When the middleware initializes a connection between Tower 1 and Tower 2, the transport negotiation proceeds as follows:

1. Tower 2 advertises its available transports: `rc` (RoCE), `dc` (Dynamic Connection), `shm` (for local V100-to-V100), `cuda_copy` (GPUDirect).
2. Tower 1 advertises: `tcp` (only option on Windows).
3. The middleware selects the best common transport. For cross-tower communication, this is `tcp` since Tower 1 cannot participate in RDMA. For Tower 2 internal communication, `rc` or `dc` is selected.
4. The `RdmaCacheConfig::transport` field records the selected transport, and all subsequent tensor operations use the negotiated path.

This negotiation is transparent to the application layer. The vLLM connector (Section 8) issues `get_tensor` and `put_gradient` calls without knowledge of the underlying transport. The same code runs whether the path is TCP (Tower 1), SoftRoCE (development), or hardware RoCE (production).

The UCX environment variable `UCX_TLS` can be used to constrain transport selection for debugging. Setting `UCX_TLS=tcp` forces TCP on both sides; setting `UCX_TLS=rc,tcp` allows the negotiation to select RC when available with TCP fallback. For our heterogeneous cluster, the recommended setting is:

```
# Tower 2 (Linux, RDMA available)
export UCX_TLS=rc,dc,tcp,cuda_copy,gdr_copy
export UCX_NET_DEVICES=mlx5_0:1

# Tower 1 (Windows, TCP only)
export UCX_TLS=tcp
```

References

[1] L. Ackroyd. Neumann: A unified tensor-based runtime. https://github.com/Shadylukin/Neumann, 2025.

[2] AMD. ROCm vLLM mixture-of-experts guide. https://rocm.blogs.amd.com/software-tools-optimization/vllm-moe-guide/README.html, 2024.

[3] D. L. Apre. Sparse autoencoders collection. https://huggingface.co/collections/dlouapre/sparse-autoencoders-675d1a2fe35e70ef80bde8f9, 2024.

[4] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[5] DeepSeek AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[6] L. Gao et al. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

[7] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training. *International Conference on Learning Representations (ICLR)*, 2018.

[8] Moonshot AI. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

[9] NVIDIA. GDRCopy: A low-latency GPU memory copy library. https://github.com/NVIDIA/gdrcopy, 2024.

[10] NVIDIA. HPC-X v2.15 documentation. https://docs.nvidia.com/networking/display/hpcxv215, 2024.

[11] NVIDIA. NCCL: NVIDIA collective communications library. https://github.com/NVIDIA/nccl, 2024.

[12] NVIDIA. NIXL: NVIDIA inference xfer library. https://github.com/NVIDIA/nixl, 2025.

[13] OpenUCX Consortium. UCC: Unified collective communication. https://github.com/openucx/ucc, 2024.

[14] OpenUCX Consortium. UCX: Unified communication X. https://github.com/openucx/ucx, 2024.

[15] B. Peng et al. Hetis: Heterogeneous GPU-cluster scheduling for efficient large language model fine-tuning. *Proceedings of the 2024 USENIX Annual Technical Conference*, 2024.

[16] G. F. Pfister. An introduction to the InfiniBand architecture. *High Performance Mass Storage and Parallel I/O*, 2001.

[17] J. Philipp. Eiffel tower Llama: SAE feature steering demo. https://huggingface.co/spaces/janphilipp/eiffel_tower_llama, 2024.

[18] Red Hat. GPUDirect RDMA on OpenShift AI: Enabling remote GPU memory access for accelerated computing. https://www.redhat.com/en/blog/gpudirect-rdma-openshift-ai-enabling-remote-gpu-memory-access-for-accelerated-computing, 2024.

[19] B. D. Rouhani et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[20] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2020.

[21] Ultra Ethernet Consortium. Ultra Ethernet Consortium 1.0 specification, 2025.

[22] vLLM Contributors. Discussion #10118: Tensor parallelism vs pipeline parallelism trade-offs. https://github.com/vllm-project/vllm/discussions/10118, 2024.

[23] vLLM Contributors. RFC: UCC integration for collective communication (PR #30930). https://github.com/vllm-project/vllm/pull/30930, 2025.